50道运维工程师面试题，速速收藏！

1、请简述OSI七层网络模型有哪些层及各自的含义?

2、在Linux的LVM分区格式下，请简述给根分区磁盘扩容的步骤?

3、讲述一下Tomcat8005、8009、8080三个端口的含义?

4、简述DNS进行域名解析的过程?

5、讲一下Keepalived的工作原理?

6、LVS、Nginx、HAproxy有什么区别？工作中你怎么选择?

7、docker的工作原理是什么，讲一下

8、docker的组成包含哪几大部分

9、docker与传统虚拟机的区别什么?

10、docker技术的三大核心概念是什么?

11、centos镜像几个G，但是docker centos镜像才几百兆，这是为什么?

12、讲一下镜像的分层结构以及为什么要使用镜像的分层结构?

13、讲一下容器的copy-on-write特性，修改容器里面的内容会修改镜像吗?

14、简单描述一下Dockerfile的整个构建镜像过程

15、Dockerfile构建镜像出现异常，如何排查?

16、Dockerfile的基本指令有哪些?

17、如何进入容器？使用哪个命令

18、什么是k8s？说出你的理解

19、k8s的组件有哪些，作用分别是什么?

20、kubelet的功能、作用是什么？（重点，经常会问）

21、kube-api-server的端口是多少？各个pod是如何访问kube-api-server的?

22、k8s中命名空间的作用是什么?

23、pod资源控制器类型有哪些?

24、nginx算法策略

25、nignx常用模块

26、如何查看并且杀死僵尸进程?

27、搜索某个用户运行的进程

28、查看某个端口正在被哪个进程使用

29、端口转发

30、查看http的并发请求数与其TCP连接状态

31、查看/var/log目录下文件数

32、linux系统启动流程

33、Linux文件类型

34、简述lvm，如何给使用lvm的/分区扩容?

35、如何在文本里面进行复制、粘贴，删除行，删除全部，按行查找和按字母查找。

36、符号链接与硬链接的区别

37、什么是正向代理?

38、什么是反向代理?

39、什么是动态资源、静态资源分离?

40、网站登陆缓慢是什么原因?

41、a与b服务器不在同一网段怎么设置?设置完还ping不通怎么排查?

42、在AB两台服务器之间通过一个服务器c做软路由使用给路由器c配置两块网卡并开启自身的路由功能

43、如果PING不通怎么排查

44、docker容器ping不通是什么原因?

45、如果一台办公室内主机无法上网(打不开网站)，请给出你的排查步骤?

46、如果我们的网站打开速度慢请说下您的排查思路?

46、如何查看二进制文件的内容

47、你是怎么备份数据的，包括数据库备份?

48、zabbix常用术语你知道几个?

49、虚拟化技术有哪些表现形式

50、修改线上业务配置文件流程

> 来源：网络技术联盟站
>
> 链接：https://www.wljslmz.cn/19875.html
>
> 版权归原作者所有，仅用于学习交流分享，侵删

1、请简述OSI七层网络模型有哪些层及各自的含义?

- 物理层：底层数据传输，比如网线、网卡标准
- 数据链路层：定义数据的基本格式，如何传输，如何标识。比如网卡MAC地址

- 网络层：定义IP编码，定义路由功能，比如不同设备的数据转发
- 传输层：端到端传输数据的基本功能，比如TCP、UDP
- 会话层：控制应用程序之间会话能力，比如不同软件数据分发给不停软件
- 表示层：数据格式标识，基本压缩加密功能。
- 应用层：各种应用软件，包括 Web 应用。

2、在Linux的LVM分区格式下，请简述给根分区磁盘扩容的步骤?

这个分3种

第一种方法:

```
growpart /dev/vda 1
resize2fs /dev/vda1
```

第二种方法:

```
partpeobe /dev/sda
resize2fs /dev/vda1
```

第三种方法:

```
fdisk /dev/sdb   # n p 1 1 回车 回车 t 8e w
pvcreate /dev/sdb1
vgextend datavg /dev/sdb1
lvextend -r -L +100%free /dev/mapper/datavg-lv01
```

3、讲述一下Tomcat8005、8009、8080三个端口的含义?

- 8005 关闭时使用
- 8009为AJP端口，即容器使用，如Apache能通过AJP协议访问Tomcat的8009端口来实现功能
- 8080 一般应用使用

4、简述DNS进行域名解析的过程?

迭代查询（返回最优结果）、递归查询（本地找DNS）用户要访问 www.baidu.com，会先找本机的host文件，再找本地设置的DNS服务器，如果也没有找到，就去网络中找根服务器，根服务器反馈结果，说只能提供一级域名服务器.cn，就去找一级域名服务器，一级域名服务器说只能提供二级域名服务器.com.cn,就去找二级域名服务器，二级域服务器只能提供三级域名服务

器.baidu.com.cn，就去找三级域名服务器，三级域名服务器正好有这个网站www.baidu.com，然后发给请求的服务器，保存一份之后，再发给客户端。

5、讲一下Keepalived的工作原理?

在一个虚拟路由器中，只有作为MASTER的VRRP(虚拟路由冗余协议)路由器会一直发送VRRP通告信息，BACKUP不会抢占MASTER，除非它的优先级更高。当MASTER不可用时(BACKUP收不到通告信息)多台BACKUP中优先级最高的这台会被抢占为MASTER。这种抢占是非常快速的(<1s)，以保证服务的连续性由于安全性考虑，VRRP包使用了加密协议进行加密。BACKUP不会发送通告信息，只会接收通告信息。

6、LVS、Nginx、HAproxy有什么区别？工作中你怎么选择?

LVS：

- 抗负载能力强、工作在第4层仅作分发之用，没有流量的产生，这个特点也决定了它在负载均衡软件里的性能最强的；无流量，同时保证了均衡器IO的性能不会受到大流量的影响；
- 工作稳定，自身有完整的双机热备方案，如LVS+Keepalived和LVS+Heartbeat；
- 应用范围比较广，可以对所有应用做负载均衡；
- 配置简单，因为没有可太多配置的东西，所以并不需要太多接触，大大减少了人为出错的几率；

LVS的缺点：

- 软件本身不支持正则处理，不能做动静分离，这就凸显了Nginx/HAProxy+Keepalived的优势。
- 如果网站应用比较庞大，LVS/DR+Keepalived就比较复杂了，特别是后面有Windows Server应用的机器，实施及配置还有维护过程就比较麻烦，相对而言，Nginx/HAProxy+Keepalived就简单多了。

Nginx：

- 工作在第7层，应用层，可以针对http应用做一些分流的策略。比如针对域名、目录结构。它的正则比HAProxy更为强大和灵活；
- Nginx对网络的依赖非常小，理论上能ping通就就能进行负载功能
- Nginx安装和配置简单
- 可以承担高的负载压力且稳定，一般能支撑超过几万次的并发量；
- Nginx不仅仅是一款优秀的负载均衡器/反向代理软件，它同时也是功能强大的Web应用服务器。Nginx在处理静态页面、特别是抗高并发方面相对apache有优势；
- Nginx作为Web反向代理加速缓存越来越成熟，速度比传统的Squid服务器更快

Nginx的缺点：

- Nginx不支持url来检测。
- Nginx仅能支持http、https和Email协议
- Nginx的Session的保持，Cookie的引导能力相对欠缺。

HAProxy：

- HAProxy是支持虚拟主机的，可以工作在4、7层(支持多网段)；
- 能够补充Nginx的一些缺点比如Session的保持，Cookie的引导等工作；
- 支持url检测后端的服务器；
- 它跟LVS一样，本身仅仅就只是一款负载均衡软件；单纯从效率上来讲HAProxy更会比Nginx有更出色的负载均衡速度，在并发处理上也是优于Nginx的；
- HAProxy可以对Mysql读进行负载均衡，对后端的MySQL节点进行检测和负载均衡，不过在后端的MySQL slaves数量超过10台时性能不如LVS；
- HAProxy的算法较多，达到8种；

工作选择：

HAproxy和Nginx由于可以做七层的转发，所以URL和目录的转发都可以做在很大并发量的时候我们就要选择LVS，像中小型公司的话并发量没那么大选择HAproxy或者Nginx足已，由于HAproxy由是专业的代理服务器配置简单，所以中小型企业推荐使用HAproxy。

7、docker的工作原理是什么，讲一下

docker是一个Client-Server结构的系统，docker守护进程运行在宿主机上，守护进程从客户端接受命令并管理运行在主机上的容器，容器是一个运行时环境，这就是我们说的集装箱。

8、docker的组成包含哪几大部分

一个完整的docker有以下几个部分组成：

- docker client，客户端，为用户提供一系列可执行命令，用户用这些命令实现跟 docker daemon 交互；
- docker daemon，守护进程，一般在宿主主机后台运行，等待接收来自客户端的请求消息；
- docker image，镜像，镜像run之后就生成为docker容器；
- docker container，容器，一个系统级别的服务，拥有自己的ip和系统目录结构；运行容器前需要本地存在对应的镜像，如果本地不存在该镜像则就去镜像仓库下载。

docker 使用客户端-服务器 (C/S) 架构模式，使用远程api来管理和创建docker容器。docker 容器通过 docker 镜像来创建。容器与镜像的关系类似于面向对象编程中的对象与类。

9、docker与传统虚拟机的区别什么?

- 传统虚拟机是需要安装整个操作系统的，然后再在上面安装业务应用，启动应用，通常需要几分钟去启动应用，而docker是直接使用镜像来运行业务容器的，其容器启动属于秒级别；
- Docker需要的资源更少，Docker在操作系统级别进行虚拟化，Docker容器和内核交互，几乎没有性能损耗，而虚拟机运行着整个操作系统，占用物理机的资源就比较多;
- Docker更轻量，Docker的架构可以共用一个内核与共享应用程序库，所占内存极小;同样的硬件环境，Docker运行的镜像数远多于虚拟机数量，对系统的利用率非常高;
- 与虚拟机相比，Docker隔离性更弱，Docker属于进程之间的隔离，虚拟机可实现系统级别隔离;
- Docker的安全性也更弱，Docker的租户root和宿主机root相同，一旦容器内的用户从普通用户权限提升为root权限，它就直接具备了宿主机的root权限，进而可进行无限制的操作。虚拟机租户root权限和宿主机的root虚拟机权限是分离的，并且虚拟机利用如Intel的VT-d和VT-x的ring-1硬件隔离技术，这种技术可以防止虚拟机突破和彼此交互，而容器至今还没有任何形式的硬件隔离;
- Docker的集中化管理工具还不算成熟，各种虚拟化技术都有成熟的管理工具，比如：VMware vCenter提供完备的虚拟机管理能力;
- Docker对业务的高可用支持是通过快速重新部署实现的，虚拟化具备负载均衡，高可用、容错、迁移和数据保护等经过生产实践检验的成熟保障机制，Vmware可承诺虚拟机99.999%高可用，保证业务连续性;
- 虚拟化创建是分钟级别的，Docker容器创建是秒级别的，Docker的快速迭代性，决定了无论是开发、测试、部署都可以节省大量时间;
- 虚拟机可以通过镜像实现环境交付的一致性，但镜像分发无法体系化，Docker在Dockerfile中记录了容器构建过程，可在集群中实现快速分发和快速部署。 from wljslmz

10、docker技术的三大核心概念是什么?

- **镜像**：镜像是一种轻量级、可执行的独立软件包，它包含运行某个软件所需的所有内容，我们把应用程序和配置依赖打包好形成一个可交付的运行环境(包括代码、运行时需要的库、环境变量和配置文件等)，这个打包好的运行环境就是image镜像文件。
- **容器**：容器是基于镜像创建的，是镜像运行起来之后的一个实例，容器才是真正运行业务程序的地方。如果把镜像比作程序里面的类，那么容器就是对象。
- **镜像仓库**：存放镜像的地方，研发工程师打包好镜像之后需要把镜像上传到镜像仓库中去，然后就可以运行有仓库权限的人拉取镜像来运行容器了。

11、centos镜像几个G，但是docker centos镜像才几百兆，这是为什么?

一个完整的Linux操作系统包含Linux内核和rootfs根文件系统，即我们熟悉的/dev、/proc/、/bin等目录。我们平时看到的centOS除了rootfs，还会选装很多软件，服务，图形桌面等，所以centOS镜像有好几个G也不足为奇。

而对于容器镜像而言，所有容器都是共享宿主机的Linux 内核的，而对于docker镜像而言，docker镜像只需要提供一个很小的rootfs即可，只需要包含最基本的命令，工具，程序库即可，所有docker镜像才会这么小。

12、讲一下镜像的分层结构以及为什么要使用镜像的分层结构?

一个新的镜像其实是从 base 镜像一层一层叠加生成的。每安装一个软件，dockerfile中使用RUM命令，就会在现有镜像的基础上增加一层，这样一层一层的叠加最后构成整个镜像。所以我们docker pull拉取一个镜像的时候会看到docker是一层层拉去的。

分层机构最大的一个好处就是 ：**共享资源**。比如：有多个镜像都从相同的 base 镜像构建而来，那么 Docker Host 只需在磁盘上保存一份 base 镜像；同时内存中也只需加载一份 base 镜像，就可以为所有容器服务了。而且镜像的每一层都可以被共享。

13、讲一下容器的copy-on-write特性，修改容器里面的内容会修改镜像吗?

我们知道，镜像是分层的，镜像的每一层都可以被共享，同时，镜像是只读的。当一个容器启动时，一个新的可写层被加载到镜像的顶部，这一层通常被称作“容器层”，“容器层”之下的都叫“镜像层”。

所有对容器的改动 – 无论添加、删除、还是修改文件，都只会发生在容器层中，因为只有容器层是可写的，容器层下面的所有镜像层都是只读的。镜像层数量可能会很多，所有镜像层会联合在一起组成一个统一的文件系统。如果不同层中有一个相同路径的文件，比如 /a，上层的 /a 会覆盖下层的 /a，也就是说用户只能访问到上层中的文件 /a。在容器层中，用户看到的是一个叠加之后的文件系统。

- **添加文件**：在容器中创建文件时，新文件被添加到容器层中。
- **读取文件**：在容器中读取某个文件时，Docker 会从上往下依次在各镜像层中查找此文件。一旦找到，立即将其复制到容器层，然后打开并读入内存。
- **修改文件**：在容器中修改已存在的文件时，Docker 会从上往下依次在各镜像层中查找此文件。一旦找到，立即将其复制到容器层，然后修改之。
- **删除文件**：在容器中删除文件时，Docker 也是从上往下依次在镜像层中查找此文件。找到后，会在

容器层中记录下此删除操作。

只有当需要修改时才复制一份数据，这种特性被称作 Copy-on-Write。可见，容器层保存的是镜像变化的部分，不会对镜像本身进行任何修改。

14、简单描述一下Dockerfile的整个构建镜像过程

- 首先，创建一个目录用于存放应用程序以及构建过程中使用到的各个文件等；
- 然后，在这个目录下创建一个Dockerfile文件，一般建议Dockerfile的文件名就是Dockerfile；
- 编写Dockerfile文件，编写指令，如，使用FORM指令指定基础镜像，COPY指令复制文件，RUN指令指定要运行的命令，ENV设置环境变量，EXPOSE指定容器要暴露的端口，WORKDIR设置当前工作目录，CMD容器启动时运行命令，等等指令构建镜像；
- Dockerfile编写完成就可以构建镜像了，使用docker build -t 镜像名:tag . 命令来构建镜像，最后一个点是表示当前目录，docker会默认寻找当前目录下的Dockerfile文件来构建镜像，如果不使用默认，可以使用-f参数来指定dockerfile文件，如：docker build -t 镜像名:tag -f /xx/xxx/Dockerfile ；
- 使用docker build命令构建之后，docker就会将当前目录下所有的文件发送给docker daemon，顺序执行Dockerfile文件里的指令，在这过程中会生成临时容器，在临时容器里面安装RUN指定的命令，安装成功后，docker底层会使用类似于docker commit命令来将容器保存为镜像，然后删除临时容器，以此类推，一层层的构建镜像，运行临时容器安装软件，直到最后的镜像构建成功。

15、Dockerfile构建镜像出现异常，如何排查?

首先，Dockerfile是一层一层的构建镜像，期间会产生一个或多个临时容器，构建过程中其实就是在临时容器里面安装应用，如果因为临时容器安装应用出现异常导致镜像构建失败，这时容器虽然被清理掉了，但是期间构建的中间镜像还在，那么我们可以根据异常时上一层已经构建好的临时镜像，将临时镜像运行为容器，然后在容器里面运行安装命令来定位具体的异常。

16、Dockerfile的基本指令有哪些?

- FROM 指定基础镜像（必须为第一个指令，因为需要指定使用哪个基础镜像来构建镜像）；
- MAINTAINER 设置镜像作者相关信息，如作者名字，日期，邮件，联系方式等；
- COPY 复制文件到镜像；
- ADD 复制文件到镜像（ADD与COPY的区别在于，ADD会自动解压tar、zip、tgz、xz等归档文件，而COPY不会，同时ADD指令还可以接一个url下载文件地址，一般建议使用COPY复制文件即可，文件在宿主机上是什么样子复制到镜像里面就是什么样子这样比较好）；

- ENV 设置环境变量；
- EXPOSE 暴露容器进程的端口，仅仅是提示别人容器使用的哪个端口，没有过多作用；
- VOLUME 数据卷持久化，挂载一个目录；
- WORKDIR 设置工作目录，如果目录不在，则会自动创建目录；
- RUN 在容器中运行命令，RUN指令会创建新的镜像层，RUN指令经常被用于安装软件包；
- CMD 指定容器启动时默认运行哪些命令，如果有多个CMD，则只有最后一个生效，另外，CMD指令可以被docker run之后的参数替换；
- ENTRYOINT 指定容器启动时运行哪些命令，如果有多个ENTRYOINT，则只有最后一个生效，另外，如果Dockerfile中同时存在CMD和ENTRYOINT，那么CMD或docker run之后的参数将被当做参数传递给ENTRYOINT；

17、如何进入容器？使用哪个命令

进入容器有两种方法：docker attach、docker exec。

18、什么是k8s？说出你的理解

K8s是kubernetes的简称，其本质是一个开源的容器编排系统，主要用于管理容器化的应用，其目标是让部署容器化的应用简单并且高效（powerful）,Kubernetes提供了应用部署，规划，更新，维护的一种机制。

说简单点：k8s就是一个编排容器的系统，一个可以管理容器应用全生命周期的工具，从创建应用，应用的部署，应用提供服务，扩容缩容应用，应用更新，都非常的方便，而且还可以做到故障自愈，所以，k8s是一个非常强大的容器编排系统。

19、k8s的组件有哪些，作用分别是什么？

k8s主要由master节点和node节点构成。master节点负责管理集群，node节点是容器应用真正运行的地方。

- master节点包含的组件有：kube-api-server、kube-controller-manager、kube-scheduler、etcd。
- node节点包含的组件有：kubelet、kube-proxy、container-runtime。

kube-api-server：以下简称api-server，api-server是k8s最重要的核心组件之一，它是k8s集群管理的统一访问入口，提供了RESTful API接口，实现了认证、授权和准入控制等安全功能；api-server还是其他组件之间的数据交互和通信的枢纽，其他组件彼此之间并不会直接通信，其他组件

对资源对象的增、删、改、查和监听操作都是交由api-server处理后，api-server再提交给etcd数据库做持久化存储，只有api-server才能直接操作etcd数据库，其他组件都不能直接操作etcd数据库，其他组件都是通过api-server间接的读取，写入数据到etcd。

kube-controller-manager：以下简称controller-manager，controller-manager是k8s中各种控制器的的管理者，是k8s集群内部的管理控制中心，也是k8s自动化功能的核心；controller-manager内部包含replication controller、node controller、deployment controller、endpoint controller等各种资源对象的控制器，每种控制器都负责一种特定资源的控制流程，而controller-manager正是这些controller的核心管理者。

kube-scheduler：以下简称scheduler，scheduler负责集群资源调度，其作用是将待调度的pod通过一系列复杂的调度算法计算出最合适的node节点，然后将pod绑定到目标节点上。shceduler会根据pod的信息，全部节点信息列表，过滤掉不符合要求的节点，过滤出一批候选节点，然后给候选节点打分，选分最高的就是最佳节点，scheduler就会把目标pod安置到该节点。

Etcd：etcd是一个分布式的键值对存储数据库，主要是用于保存k8s集群状态数据，比如，pod，service等资源对象的信息；etcd可以是单个也可以有多个，多个就是etcd数据库集群，etcd通常部署奇数个实例，在大规模集群中，etcd有5个或7个节点就足够了；另外说明一点，etcd本质上可以不与master节点部署在一起，只要master节点能通过网络连接etcd数据库即可。

kubelet：每个node节点上都有一个kubelet服务进程，kubelet作为连接master和各node之间的桥梁，负责维护pod和容器的生命周期，当监听到master下发到本节点的任务时，比如创建、更新、终止pod等任务，kubelet 即通过控制docker来创建、更新、销毁容器； 每个kubelet进程都会在api-server上注册本节点自身的信息，用于定期向master汇报本节点资源的使用情况。

kube-proxy：kube-proxy运行在node节点上，在Node节点上实现Pod网络代理，维护网络规则和四层负载均衡工作，kube-proxy会监听api-server中从而获取service和endpoint的变化情况，创建并维护路由规则以提供服务IP和负载均衡功能。简单理解此进程是Service的透明代理兼负载均衡器，其核心功能是将到某个Service的访问请求转发到后端的多个Pod实例上。

container-runtime：容器运行时环境，即运行容器所需要的一系列程序，目前k8s支持的容器运行时有很多，如docker、rkt或其他，比较受欢迎的是docker，但是新版的k8s已经宣布弃用docker。

20、kubelet的功能、作用是什么？（重点，经常会问）

kubelet部署在每个node节点上的，它主要有4个功能：

- 节点管理。kubelet启动时会向api-server进行注册，然后会定时的向api-server汇报本节点信息状态，资源使用状态等，这样master就能够知道node节点的资源剩余，节点是否失联等等相关的信息

了。master知道了整个集群所有节点的资源情况，这对于 pod 的调度和正常运行至关重要。

- **pod管理**。kubelet负责维护node节点上pod的生命周期，当kubelet监听到master的下发到自己节点的任务时，比如要创建、更新、删除一个pod，kubelet 就会通过CRI（容器运行时接口）插件来调用不同的容器运行时来创建、更新、删除容器；常见的容器运行时有docker、containerd、rkt等等这些容器运行时，我们最熟悉的就是docker了，但在新版本的k8s已经弃用docker了，k8s1.24版本中已经使用containerd作为容器运行时了。
- **容器健康检查**。pod中可以定义启动探针、存活探针、就绪探针等3种，我们最常用的就是存活探针、就绪探针，kubelet 会定期调用容器中的探针来检测容器是否存活，是否就绪，如果是存活探针，则会根据探测结果对检查失败的容器进行相应的重启策略；
- Metrics Server资源监控。在node节点上部署Metrics Server用于监控node节点、pod的CPU、内存、文件系统、网络使用等资源使用情况，而kubelet则通过Metrics Server获取所在节点及容器的上的数据。

21、kube-api-server的端口是多少？各个pod是如何访问kube-api-server的?

kube-api-server的端口是8080和6443，前者是http的端口，后者是https的端口，以我本机使用kubeadm安装的k8s为例：

在命名空间的kube-system命名空间里，有一个名称为kube-api-master的pod，这个pod就是运行着kube-api-server进程，它绑定了master主机的ip地址和6443端口，但是在default命名空间下，存在一个叫kubernetes的服务，该服务对外暴露端口为443，目标端口6443，这个服务的ip地址是clusterip地址池里面的第一个地址，同时这个服务的yaml定义里面并没有指定标签选择器，也就是说这个kubernetes服务所对应的endpoint是手动创建的，该endpoint也是名称叫做kubernetes，该endpoint的yaml定义里面代理到master节点的6443端口，也就是kube-api-server的IP和端口。这样一来，其他pod访问kube-api-server的整个流程就是：pod创建后嵌入了环境变量，pod获取到了kubernetes这个服务的ip和443端口，请求到kubernetes这个服务其实就是转发到了master节点上的6443端口的kube-api-server这个pod里面。

22、k8s中命名空间的作用是什么?

amespace是kubernetes系统中的一种非常重要的资源，namespace的主要作用是用来实现多套环境的资源隔离，或者说是多租户的资源隔离。

k8s通过将集群内部的资源分配到不同的namespace中，可以形成逻辑上的隔离，以方便不同的资源进行隔离使用和管理。不同的命名空间可以存在同名的资源，命名空间为资源提供了一个作用域。

可以通过k8s的授权机制，将不同的namespace交给不同的租户进行管理，这样就实现了多租户的资源隔离，还可以结合k8s的资源配额机制，限定不同的租户能占用的资源，例如CPU使用量、内存使用量等等来实现租户可用资源的管理。

23、pod资源控制器类型有哪些?

- Deployments：Deployment为Pod和ReplicaSet提供声明式的更新能力。
- ReplicaSet：ReplicaSet的目的是维护一组在任何时候都处于运行状态的Pod副本的稳定集合。因此，它通常用来保证给定数量的、完全相同的Pod的可用性。
- StatefulSets：和Deployment类似，StatefulSet管理基于相同容器规约的一组Pod。但和Deployment不同的是，StatefulSet为它们的每个Pod维护了一个有粘性的ID。这些Pod是基于相同的规约来创建的，但是不能相互替换：无论怎么调度，每个Pod都有一个永久不变的ID。
- DaemonSet：DaemonSet确保全部（或者某些）节点上运行一个Pod的副本。当有节点加入集群时，也会为他们新增一个Pod。当有节点从集群移除时，这些Pod也会被回收。删除DaemonSet将会删除它创建的所有Pod。
- Jobs：Job会创建一个或者多个Pod，并将继续重试Pod的执行，直到指定数量的Pod成功终止。随着Pod成功结束，Job跟踪记录成功完成的Pod个数。当数量达到指定的成功个数阈值时，任务（即Job）结束。删除Job的操作会清除所创建的全部Pod。挂起Job的操作会删除Job的所有活跃Pod，直到Job被再次恢复执行。
- Automatic Clean-up for Finished Jobs：TTL-after-finished控制器提供了一种TTL机制来限制已完成执行的资源对象的生命周期。TTL控制器目前只处理Job。
- CronJob：一个CronJob对象就像crontab(crontable)文件中的一行。它用Cron格式进行编写，并周期性地在给定的调度时间执行Job。
- ReplicationController：ReplicationController确保在任何时候都有特定数量的Pod副本处于运行状态。换句话说，ReplicationController确保一个Pod或一组同类的Pod总是可用的。

24、nginx算法策略

轮询（默认）

加权轮询（轮询+weight）

ip_hash

每一个请求的访问IP，都会映射成一个hash，再通过hash算法（hash值%node_count），分配到不同的后端服务器，访问ip相同的请求会固定访问同一个后端服务器，这样可以做到会话保持，解决session同步问题。

least_conn（最少连接）

使用最少连接的负载平衡，nginx将尝试不会使繁忙的应用程序服务器超载请求过多，而是将新请求分发给不太繁忙的服务器。

25、nignx常用模块

- upstream
- rewrite
- location
- proxy_pass

26、如何查看并且杀死僵尸进程?

top —> task (line)—> zombie.

把父进程杀掉，父进程死后，过继给1号进程init，init 始终负责清理僵尸进程，它产生的所有僵尸进程跟着消失；如果你使用kill ，一般都不能杀掉 defunct进程.。用了kill –15,kill –9以后 之后反而会多出更多的僵尸进程。

27、搜索某个用户运行的进程

```
pgrep -au neteagle
```

28、查看某个端口正在被哪个进程使用

```
lsof -i :[port]
```

29、端口转发

```
iptables -t nat -A PREROUTING -d 10.0.0.8 -p tcp --dport 80 -j REDIRECT --to-ports 8080
```

30、查看http的并发请求数与其TCP连接状态

```
etstat-n|awk'/^tcp/{++b[$NF]}END{for(ainb)printa,b[a]}'
```

31、查看/var/log目录下文件数

```
ls/var/log/-lR|grep"^-"|wc-l
```

32、linux系统启动流程

- 第一步：开机自检，加载BIOS
- 第二步：读取M B R
- 第三步：Boot Loader grub引导菜单
- 第四步：加载kernel内核
- 第五步：init进程依据inittab文件夹来设定运行级别
- 第六步：init进程执行rc.sysinit
- 第七步：启动内核模块
- 第八步：执行不同运行级别的脚本程序
- 第九步：执行/etc/rc.d/rc.lo

33、Linux文件类型

- –：常规文件，即file
- d：目录文件
- b：block device 即块设备文件，如硬盘;支持以block为单位进行随机访问
- c：character device 即字符设备文件，如键盘支持以character为单位进行线性访问
- l：symbolic link 即符号链接文件（关注微信公众号：网络技术联盟站），又称软链接文件
- p：pipe 即命名管道文件
- s：socket 即套接字文件，用于实现两个进程进行通信

34、简述lvm，如何给使用lvm的/分区扩容?

功能：可以对磁盘进行动态管理。动态按需调整大小

概念：

- PV 物理卷：物理卷在逻辑卷管理中处于最底层，它可以是实际物理硬盘上的分区，也可以是整个物

理硬盘，也可以是raid设备。

- **VG 卷组**：卷组建立在物理卷之上，一个卷组中至少要包括一个物理卷，在卷组建立之后可动态添加物理卷到卷组中。一个逻辑卷管理系统工程中可以只有一个卷组，也可以拥有多个卷组。
- **LV 逻辑卷**：逻辑卷建立在卷组之上，卷组中的未分配空间可以用于建立新的逻辑卷，逻辑卷建立后可以动态地扩展和缩小空间。系统中的多个逻辑卷可以属于同一个卷组，也可以属于不同的多个卷组。

给/分区扩容步骤：

- 添加磁盘
- 使用fdisk命令对新增加的磁盘进行分区
- 分区完成后修改分区类型为lvm
- 使用pvcreate创建物理卷
- 使用vgextend命令将新增加的分区加入到根目录分区中
- 使用lvextend命令进行扩容
- 使用xfs_growfs调整卷分区大小

35、如何在文本里面进行复制、粘贴，删除行，删除全部，按行查找和按字母查找。

以下操作全部在vi/vim命令行状态操作，不要在编辑状态操作：

- 在文本里 移动到想要复制的行按yy想复制到哪就移动到哪，然后按P就黏贴了
- 删除行　移动到改行 按dd
- 删除全部dG这里注意G一定要大写
- 按行查找　:90 这样就是找到第90行
- 按字母查找 /path 这样就是找到path这个单词所在的位置，文本里可能存在多个，多次查找会显示在不同的位置。

36、符号链接与硬链接的区别

- 我们可以把符号链接，也就是软连接 当做是 windows系统里的 快捷方式。
- 硬链接 就好像是 又复制了一份.
- ln 3.txt 4.txt　这是硬链接，相当于复制，不可以跨分区，但修改3,4会跟着变，若删除3,4不受任何影响。
- ln –s 3.txt 4.txt　这是软连接，相当于快捷方式。修改4,3也会跟着变，若删除3,4就坏掉了。不可以

用了。

37、什么是正向代理?

一个位于客户端和原始服务器(origin server)之间的服务器，为了从原始服务器取得内容，客户端向代理发送一个请求并指定目标(原始服务器)，然后代理向原始服务器转交请求并将获得的内容返回给客户端。

客户端才能使用正向代理。正向代理总结就一句话：代理端代理的是客户端。例如说：我们使用的 OpenVPN 等等。

38、什么是反向代理?

反向代理（Reverse Proxy）方式，是指以代理服务器来接受 Internet上的连接请求，然后将请求，发给内部网络上的服务器并将从服务器上得到的结果返回给 Internet 上请求连接的客户端，此时代理服务器对外就表现为一个反向代理服务器。

反向代理总结就一句话：代理端代理的是服务端。

39、什么是动态资源、静态资源分离?

动态资源、静态资源分离，是让动态网站里的动态网页根据一定规则把不变的资源和经常变的资源区分开来，动静资源做好了拆分以后我们就可以根据静态资源的特点将其做缓存操作，这就是网站静态化处理的核心思路。

动态资源、静态资源分离简单的概括是：动态文件与静态文件的分离。

在我们的软件开发中，有些请求是需要后台处理的（如：.jsp,.do 等等），有些请求是不需要经过后台处理的（如：css、html、jpg、js 等等文件），这些不需要经过后台处理的文件称为静态文件，否则动态文件。

因此我们后台处理忽略静态文件。这会有人又说那我后台忽略静态文件不就完了吗？当然这是可以的，但是这样后台的请求次数就明显增多了。在我们对资源的响应速度有要求的时候，我们应该使用这种动静分离的策略去解决动、静分离将网站静态资源（HTML，JavaScript，CSS，img等文件）与后台应用分开部署，提高用户访问静态代码的速度，降低对后台应用访问

这里我们将静态资源放到 Nginx 中，动态资源转发到 Tomcat 服务器中去。

当然，因为现在七牛、阿里云等 CDN 服务已经很成熟，主流的做法，是把静态资源缓存到 CDN 服务中，从而提升访问速度。

相比本地的 Nginx 来说，CDN 服务器由于在国内有更多的节点，可以实现用户的就近访问。并且，CDN 服务可以提供更大的带宽，不像我们自己的应用服务，提供的带宽是有限的。

40、网站登陆缓慢是什么原因?

- 网络带宽，这是一个很常见的瓶颈。
- cpu、硬盘、内存配置过低，服务器负载不起来。
- 网站的开发代码不够完善，例如mysql语句没有进行优化，导致数据库的读写相当耗费时间。
- 数据库的瓶颈。当我们的数据库的数据变得越来越多的时候，那么对于数据库的读写压力肯定会变大。

41、a与b服务器不在同一网段怎么设置?设置完还ping不通怎么排查?

- AB服务器不在同一个网段
- 首先把不同IP段的服务器划分给不同的vlan
- 在通过通过三层交换机添加虚拟IP路由实在不同网段的vlan的连接

42、在AB两台服务器之间通过一个服务器c做软路由使用给路由器c配置两块网卡并开启自身的路由功能

```
vi /etc/sysconfig/network-scripts/ifcfg-eth0
```

查看网卡状况IP –a –s 网卡的名字

A服务器设置相关网卡信息

```
子网掩码：255.255.255.0
IP=10.0.0.1
网关=10.0.0.254
重启网卡生效
查看路由信息
route –n
添加对应路由
route add –net 10.0.1.0/24 gw 10.0.0.11
```

B服务器的设置相关信息

```
IP=10.0.1.10
网关10.0.1.254
重启网卡生效
route -n
添加对应的路由
route add -net 10.0.0.0/24 gw 10.0.1.11
```

C服务器的两块网卡

```
 网卡1
 IP=10.0.0.11
 网关=10.0.0.254


 网卡2
 IP=10.0.1.11
 网关=10.0.1.254
 重启网卡生效
 route -n
 vi /etc/sysctl.conf
 net.ipv4.ip_forword = 1
```

43、如果PING不通怎么排查

- 首先先看看是不是网路接口故障水晶头或是网卡接口接触不良造成，其次检查交换机和路由等网络设备是有故障
- 是否关闭了防火墙和selinux机制
- 然后查看网卡和路由和网关是否配置正确

44、docker容器ping不通是什么原因?

ifconfig 查看一下docker0网桥，ping一下网桥看看是否通，有可能是网桥配置问题

weave路由器端口6783

- 安装docker容器的服务器没有关闭防火墙(访问一下安装docker物理机的，是否能访问，如果不能访问就变不能访问docker)
- docker在创建镜像的时候没有做端口映射(出现这种情况能访问物理机不能访问docker)使用 dockers ps 查看镜像的端口映射情况
- 端口映射不正确
- 查看网络配置ping网桥看是否能ping通，有可能是网桥的原因

45、如果一台办公室内主机无法上网(打不开网站)，请给出你的排查步骤?

- 首先确定物理链路是否联通正常。
- 查看本机IP，路由，DNS的设置情况是否达标。
- telnet检查服务器的WEB有没有开启以及防火墙是否阻拦。
- ping一下网关，进行最基础的检查，通了，表示能够到达服务器。
- 测试到网关或路由器的通常情况，先测网关，然后再测路由器一级一级的测试。
- 测试ping公网ip的通常情况(记住几个外部IP),
- 测试DNS的通畅。ping出对应IP。
- 通过以上检查后，还在网管的路由器上进行检查。

46、如果我们的网站打开速度慢请说下您的排查思路?

判断原因

首先我要以用户的身份登录我们的网站，判断问题出现在我们自身原因，还是用户那边的原因。

如果是用户问题有以下几个原因:

- 用户那边的带宽
- 用户的浏览器器版本低，安装插件太多
- 中毒和电脑里的垃圾文件过多
- 用户主机的主机的性能和操作系统

如果是我们的网站自身问题有一下几个原因

- 网络带宽
- 服务器的cpu、硬盘、内存过低服务器负载不起来也就是说服务器自身的性能方面
- 网站代码不够完善。如mysql语句没有进行优化导致数据库读写耗时
- 服务器未开启图片压缩
- 网页台下
- 死连接过多插件使用及js文件调用频繁网站服务器的速度或是租用空间所在的服务器速度

解决思路

1、检测服务器速度的快慢

- ping命令查看连接到服务器的时间和丢包情况(ping 测试网址的)
- 查看丢包率(1000个包没有丢一个是最理想的、一般一个速度好的机房丢包率不超过1%)
- ping值要小同城电信adsl ping平均值绝对不能超过20，一般都在10，跨省的平均值20-40属于正常

- ping值要均匀最小值和最大值相差太大说明路由不稳定的表现

2、查看服务器自身性能

查看cpu的使用率uptime

查看内存情况 free –m

查看I/O读写iostat 磁盘I/O读写等看看是那个进程大量占用系统资源导致我的服务器变慢

3、看看访问最多的URL和IP有什么特征，如果是恶意URL和IP就把他屏蔽掉如果是善意的就限流有可能是CDN回源量大造成网站无法访问

4、查看同台服务器上其他网站的打开速度，可以通过查询工具查看和自己在同一台服务器上的网站个数和网址可以看他们打开快慢

5、电信和联通互访的问题

如果是空间打开时快时慢，有时打不开那就是空间不稳定找空间商解决或是换空间伤，如果是有的地方快有的地方慢应该是网络线路问题，比如电信用户访问放在联通服务器上的网站，联通用户访问放在电信服务器上的网站，解决办吧是：使用双线空间或是多线空间

6、从网站自身的原因

- 网站的程序设计结构是否合理是否由于幻灯片代码影响网站打开速度(找程序设计相关人士解决)
- 网页的设计结构和代码错误(请专业人士进行修改)
- 网页的内容如：大尺寸图片、大尺寸flash、过多的引用其他网站内容，如果被引用内容的网站速度慢，也影响自身网站把。譬如友情连接可以把对方 的图片放到自己网站上

解决办法

- 优化图片，限制图片大小尺寸，降低图片质量，减少图片数量
- 限定图片的格式：jpg，png，gif
- 减少http的请求数(当打开网页时浏览器会发出很多对象请求，每个对象的加载都会有所延时，如果网页上的对象很多就会花费大量的时间，去除不必要的对象，将临近的图片合成一张，合并css文件) f r o m： w l j s l m z

46、如何查看二进制文件的内容

我们一般通过 hexdump 命令 来查看二进制文件的内容。

hexdump –C XXX(文件名） –C 是参数 不同的参数有不同的意义

–C 是比较规范的 十六进制和 ASCII 码显示

–c 是单字节字符显示

–b 单字节八进制显示

–o 是双字节八进制显示

–d 是双字节十进制显示

–x 是双字节十六进制显示

等等等等

47、你是怎么备份数据的，包括数据库备份?

在生产环境下，不管是应用数据、还是数据库数据首先在部署的时候就会有主从架构、或者集群，这本身就是属于数据的热备份；其实考虑冷备份，用专门一台服务器做为备份服务器，比如可以用rsync+inotify配合计划任务来实现数据的冷备份，如果是发版的包备份，正常情况下有台发布服务器，每次发版都会保存好发版的包。

48、zabbix常用术语你知道几个?

- 主机（host）：要监控的网络设备，可由IP或DNS名称指定；
- 主机组（hostgroup）：主机的逻辑容器，可以包含主机和模板，但同一个组织内的主机和模板不能互相链接；主机组通常在给用户或用户组指派监控权限时使用；
- 监控项（item）：一个特定监控指标的相关的数据；这些数据来自于被监控对象；item是zabbix进行数据收集的核心，相对某个监控对象，每个item都由"key"标识；
- 触发器（trigger）：一个表达式，用于评估某监控对象的特定item内接收到的数据是否在合理范围内，也就是阈值；接收的数据量大于阈值时，触发器状态将从"OK"转变为"Problem"，当数据再次恢复到合理范围，又转变为"OK"；
- 事件（event）：触发一个值得关注的事情，比如触发器状态转变，新的agent或重新上线的agent的自动注册等；
- 动作（action）：指对于特定事件事先定义的处理方法，如发送通知，何时执行操作；
- 报警升级（escalation）：发送警报或者执行远程命令的自定义方案，如每隔5分钟发送一次警报，共发送5次等；
- 媒介（media）：发送通知的手段或者通道，如Email、Jabber或者SMS等；
- 通知（notification）：通过选定的媒介向用户发送的有关某事件的信息； 远程命令（remote command）：预定义的命令，可在被监控主机处于某特定条件下时自动执行；
- 模板（template）：用于快速定义被监控主机的预设条目集合，通常包含了item、trigger、graph、screen、application以及low-level
- discovery rule；模板可以直接链接至某个主机； 应用（application）：一组item的集合；

- web场景（webscennario）：用于检测web站点可用性的一个活多个HTTP请求； 前端（frontend）：Zabbix的web接口；

49、虚拟化技术有哪些表现形式

- 完全拟化技术：通过软件实现对操作系统的资源再分配，比较成熟，完全虚拟化代表技术：KVM、ESXI、Hyper-V。
- 半虚拟化技术：通过代码修改已有的系统，形成一种新的可虚拟化的系统，调用硬件资源去安装多个系统，整体速度上相对高一点，半虚拟化代表技术：Xen。
- 轻量级虚拟化：介于完全虚拟化、半虚拟化之间，轻量级虚拟化代表技术：Docker。

50、修改线上业务配置文件流程

- 先告知运维经理和业务相关开发人员
- 在测试环境测试，并备份之前的配置文件
- 测试无误后修改生产环境配置
- 观察生产环境是否正常，是否有报警
- 完成配置文件更改